

| 编 号    | 题 号          | 标 题                    | 知 识 点      | 代 码 作 者 |
|--------|--------------|------------------------|------------|---------|
| 例 4-44 | SPOJ MKTHNUM | K-th Number            | 可持久化线段树    | 陈锋      |
| 例 4-45 | HDU 4757     | Tree                   | 可持久化 Trie  | 陈锋      |
| 例 4-46 | IPSC 2011    | Grid surveillance      | 可持久化树状数组   | 陈锋      |
| 例 4-47 | UVa 12538    | Version Controlled IDE | 可持久化 Treap | 陈锋      |

## 4.9 嵌套和分块数据结构

除前文讨论的数据结构外，还有两种“非主流”的数据结构设计方法：嵌套和分块。它们虽然较少出现在教科书中，但思想精巧，实用性高。下面给出一些与嵌套和分块有关的经典题目及代码实现。

### 例4-48 【二维线段树】人口普查 (Census, UVa 11297)

你的任务是维护一个 $n$ 行 $m$ 列 ( $1 \leq n, m \leq 500$ ) 的数字矩阵，要求支持 $Q$  ( $Q \leq 40\,000$ ) 个操作。操作分为以下两种类型。

- ☐ q  $x_1\ y_1\ x_2\ y_2$ : 查询所有满足 $x_1 \leq x \leq x_2, y_1 \leq y \leq y_2$ 的格子 $(x,y)$ 的最大值和最小值。
- ☐ c  $x\ y\ v$ : 把格子 $(x,y)$ 的值修改成 $v$ 。

对于每个q查询，依次输出最大值和最小值。

#### 【代码实现】

```

// 陈锋
#include <stdio.h>
#include <algorithm>
#include <cstring>
using namespace std;
const int NN = 508, INF = 1e9;
struct SegTree2D {
    struct Node {
        int Max, Min;
        void update(const Node& nd) {
            Max = max(Max, nd.Max), Min = min(Min, nd.Min);
        }
    } NS[NN][NN * 4]; // 第一维表示对应的矩阵的行编号

    Node gAns;
    void maintain(int c, int o) {
        Node &nd = NS[c][o], ld = NS[c][2 * o], rd = NS[c][2 * o + 1];
        nd.Max = max(ld.Max, rd.Max), nd.Min = min(ld.Min, rd.Min);
    }
}

```

```

void build(int c, int o, int l, int r) {
    Node& nd = NS[c][o];
    if (l == r) {
        scanf("%d", &nd.Min), nd.Max = nd.Min;
        return;
    }
    int mid = (l + r) / 2, lc = o * 2, rc = o * 2 + 1;
    build(c, lc, l, mid), build(c, rc, mid + 1, r);
    maintain(c, o);
}

void query(int c, int o, int l, int r, int qL, int qR) {
    if (l == qL && r == qR) {
        qAns.update(NS[c][o]);
        return;
    }
    int qM = (qL + qR) / 2, lc = o * 2, rc = o * 2 + 1;
    if (qM >= r)
        query(c, lc, l, r, qL, qM);
    else if (qM < l)
        query(c, rc, l, r, qM + 1, qR);
    else
        query(c, lc, l, qM, qL, qM), query(c, rc, qM + 1, r, qM + 1, qR);
}

void modify(int c, int x, int val, int o, int l, int r) {
    Node& nd = NS[c][o];
    if (l == r && l == x) {
        nd.Max = nd.Min = val;
        return;
    }
    int m = (l + r) / 2, lc = o * 2, rc = o * 2 + 1;
    if (m >= x)
        modify(c, x, val, lc, l, m);
    else if (m < x)
        modify(c, x, val, rc, m + 1, r);
    maintain(c, o);
}

};

SegTree2D ST;

int main() {
    char op[10];
    for (int m, n, x1, y1, x2, y2, v; scanf("%d", &n) != EOF;) {
        for (int x = 1; x <= n; x++) ST.build(x, 1, 1, n);
        scanf("%d", &m);
        while (m--) {
            scanf("%s", op);
            if (op[0] == 'q') {

```

```

    ST.qAns.Max = -INF, ST.qAns.Min = INF;
    scanf("%d%d%d%d", &x1, &y1, &x2, &y2);
    for (int x = x1; x <= x2; x++) ST.query(x, 1, y1, y2, 1, n);
    printf("%d %d\n", ST.qAns.Max, ST.qAns.Min);
}
if (op[0] == 'c')
    scanf("%d%d%d", &x1, &y1, &v), ST.modify(x1, y1, v, 1, 1, n);
}
}
return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 6.2 节例题 4
注意：本题通过一个二维数组，将普通线段树改造成二维线段树
同类题目：Mosaic, HDU 4819
*/

```

#### 例4-49 【树状数组套静态BST】“动态”逆序对（“Dynamic” Inversion, UVa 11990）

给定一个 $1 \sim n$ 的排列 $A$ ，要求按照某种顺序删除一些数（其他数顺序不变），输出每次删除之前逆序对的数目。所谓逆序对数，就是满足 $i < j$ 且 $A[i] > A[j]$ 的有序对 $(i, j)$ 的数目。

输入包含多组数据。每组数据：第一行为两个整数 $n$ 和 $m$ （ $1 \leq n \leq 2 \times 10^5$ ， $1 \leq m \leq 10^5$ ）；接下来的 $n$ 行表示初始排列；接下来的 $m$ 行按顺序给出要删除的整数，每个整数保证不会删除两次。输入结束标志为文件结束符（EOF）。输入文件大小不超过5MB。

对于每次删除，输出删除之前的逆序对数。

#### 【代码实现】

```

// 刘汝佳
#include <cstdio>
#include <vector>
#include <algorithm>
#include <cassert>
using namespace std;

inline int lowbit(int x) { return x&-x; }

struct Node {
    Node *ch[2];          // 左右子树
    int v;                 // 值
    int s;                 // 结点总数（有删除标记的结点未统计在内）
    int d;                 // 删除标记
    Node():d(0) {}
    int ch_s(int d) { return ch[d] == NULL ? 0 : ch[d]->s; }
};

// 名次树，懒删除实现
struct RankTree {
    int n, next;
    int *v;

```

```

Node *nodes, *root;
RankTree(int n, int* A):n(n) {
    nodes = new Node[n];
    next = 0;
    v = new int[n];
    for(int i = 0; i < n; i++) v[i] = A[i];
    sort(v, v+n);
    root = build(0, n-1);
    delete[] v;
}

Node* build(int L, int R) {
    if(L > R) return NULL;
    int M = L + (R-L) / 2;
    int u = next++;
    nodes[u].v = v[M];
    nodes[u].ch[0] = build(L, M-1);
    nodes[u].ch[1] = build(M+1, R);
    nodes[u].s = nodes[u].ch_s(0) + nodes[u].ch_s(1) + 1;
    return &nodes[u];
}

// type = 0: 统计比 v 小的元素个数
// type = 1: 统计比 v 大的元素个数
int count(int v, int type) {
    Node* u = root;
    int cnt = 0;
    while(u != NULL) {
        if(u->v == v) { cnt += u->ch_s(type); break; }
        int c = (v < u->v ? 0 : 1);
        if(c != type) cnt += u->s - u->ch_s(c);
        u = u->ch[c];
    }
    return cnt;
}

// 要保证 v 在树中且尚未被删除
void erase(int v) {
    Node* u = root;
    while(u != NULL) {
        u->s--;
        if(u->v == v) { assert(u->d == 0); u->d = 1; return; }
        int c = (v < u->v ? 0 : 1);
        u = u->ch[c];
    }
    assert(0);
}

~RankTree() {

```

```

        delete[] nodes;
    }
};

// 嵌套名次树的 Fenwick 树
struct FenwickRankTree {
    int n;
    vector<RankTree*> C;

    void init(int n, int* A) {
        this->n = n;
        C.resize(n+1);          // 存放在 C[1]~C[n]
        for(int i = 1; i <= n; i++) {
            C[i] = new RankTree(lowbit(i), A+i-lowbit(i)+1);
        }
    }

    void clear() { for(int i = 1; i <= n; i++) delete C[i]; }

    // 统计 A[1], A[2], ..., A[x] 有多少个元素比 v 大 (x ≤ n)
    int count(int x, int v, int type) {
        int ret = 0;
        while(x > 0) {
            ret += C[x]->count(v, type); x -= lowbit(x);
        }
        return ret;
    }

    // 删除 A[x]=v
    void erase(int x, int v) {
        while(x <= n) {
            C[x]->erase(v); x += lowbit(x);
        }
    }
};

// 普通 Fenwick 树
struct FenwickTree {
    int n;
    vector<int> C;

    void init(int n) {
        this->n = n;
        C.resize(n+1);
        fill(C.begin(), C.end(), 0);
    }

    // 计算 A[1]+A[2]+...+A[x] (x ≤ n)
    int sum(int x) {

```

```

int ret = 0;
while(x > 0) {
    ret += C[x]; x -= lowbit(x);
}
return ret;
}

// A[x] += d (1 ≤ x ≤ n)
void add(int x, int d) {
    while(x ≤ n) {
        C[x] += d; x += lowbit(x);
    }
}

};

const int maxn = 200000 + 5;
const int maxm = 100000 + 5;
typedef long long LL;

int n, m, A[maxn], B[maxn], pos[maxn];
FenwickRankTree frt;
FenwickTree f; // 用来求逆序对数以及求已删除的元素有多少个比 v 小

LL inversion_pairs() {
    LL ans = 0;
    f.init(n);
    for(int i = n; i >= 1; i--) {
        ans += f.sum(A[i]-1);
        f.add(A[i], 1);
    }
    return ans;
}

int main() {
    while(scanf("%d%d", &n, &m) == 2) {
        for(int i = 1; i ≤ n; i++) {
            scanf("%d", &A[i]);
            pos[B[i] = A[i]] = i;
        }
        LL cnt = inversion_pairs();
        frt.init(n, A);
        f.init(n);
        for(int i = 0; i < m; i++) {
            printf("%lld\n", cnt);
            int x;
            scanf("%d", &x);
            f.add(x, 1);
            int a = frt.count(pos[x]-1, x, 1); // x 左边有 a 个数比 x 大
            int b = x-1; // 一共有 x-1 个数比 x 小
        }
    }
}

```



```

    int c = f.sum(x-1);           // 删了 c 个比 x 小的数
    int d = frt.count(pos[x]-1, x, 0); // 现在左边有 d 个比 x 小的数
    b -= c + d;                  // 还剩 b 个
    cnt -= a + b;                // 逆序对减少 a+b 个
    frt.erase(pos[x], x);
}
}
return 0;
}
/*
算法分析请参考:《算法竞赛入门经典——训练指南》升级版 6.2 节例题 5
同类题目: CRB and Queries, HDU 5412
*/

```

#### 例 4-50 【分块数组】数组变换 (Array Transformation, UVa 12003)

输入一个数组  $A[1, \dots, n]$  和  $m$  ( $1 \leq n \leq 300\,000$ ,  $1 \leq m \leq 50\,000$ ,  $1 \leq A[i] \leq u \leq 10^9$ ) 条指令, 你的任务是对数组进行变换, 输出最终结果。每条指令形如  $(L, R, v, p)$ , 表示先统计出  $A[L], A[L+1], \dots, A[R]$  中严格小于  $v$  的元素个数  $k$ , 然后把  $A[p]$  修改成  $uk/(R-L+1)$ 。这里的除法为整数除法 (即忽略小数部分)。

输出  $n$  行, 每行为一个整数, 即变换后的最终数组。

#### 【代码实现】

```

// 刘汝佳
#include <cstdio>
#include <algorithm>
using namespace std;

const int maxn = 300000 + 10;
const int SIZE = 4096;

int n, m, u, A[maxn], block[maxn/SIZE+1][SIZE];

void init() {
    scanf("%d%d%d", &n, &m, &u);
    int b = 0, j = 0;
    for(int i = 0; i < n; i++) {
        scanf("%d", &A[i]);
        block[b][j] = A[i];
        if(++j == SIZE) { b++; j = 0; }
    }
    for(int i = 0; i < b; i++) sort(block[i], block[i]+SIZE);
    if(j) sort(block[b], block[b]+j);
}

int query(int L, int R, int v) {
    int lb = L/SIZE, rb = R/SIZE; // L 和 R 所在块编号
    int k = 0;
    if(lb == rb) {
        for(int i = L; i <= R; i++) if(A[i] < v) k++;
    } else {
        for(int i = L; i < (lb+1)*SIZE; i++) if(A[i] < v) k++; // 第一块
        for(int i = rb*SIZE; i <= R; i++) if(A[i] < v) k++; // 最后一块
        for(int b = lb+1; b < rb; b++) // 中间的完整块
            k += lower_bound(block[b], block[b]+SIZE, v) - block[b];
    }
    return k;
}

void change(int p, int x) {
    if(A[p] == x) return;
    int old = A[p], pos = 0, *B = &block[p/SIZE][0]; // B 就是 p 所在的块
    A[p] = x;

    while(B[pos] < old) pos++; B[pos] = x; // 找到 x 在块中的位置
    if(x > old) // x 太大, 往后交换
        while(pos < SIZE-1 && B[pos] > B[pos+1]) { swap(B[pos+1], B[pos]); pos++; }
    else // 往前交换
        while(pos > 0 && B[pos] < B[pos-1]) { swap(B[pos-1], B[pos]); pos--; }
}

int main() {
    init();
    while(m--) {
        int L, R, v, p;
        scanf("%d%d%d%d", &L, &R, &v, &p); L--; R--; p--;
        int k = query(L, R, v);
        change(p, (long long)u * k / (R-L+1));
    }
    for(int i = 0; i < n; i++) printf("%d\n", A[i]);
    return 0;
}
/*
算法分析请参考:《算法竞赛入门经典——训练指南》升级版 6.2 节例题 6
*/

```

#### 例4-51 【树分块】王室联邦 (SCOI 2005, 牛客NC 208301)

国王想把他的国家划分成若干个省，每个省都由王室联邦的一个成员来管理。国家有编号为 $1 \sim N$  ( $1 \leq N \leq 1000$ ) 的 $N$ 个城市。一些城市之间有道路相连，任意两个不同的城市之间有且仅有一条直接或间接的道路。为了防止管理太过分散，每个省至少要有 $B$  ( $1 \leq B \leq N$ ) 个城市；为了能有效地管理，每个省最多只有 $3B$ 个城市。

每个省必须有一个省会，这个省会可以位于省内，也可以位于省外。该省任意一个城市到达省会所经过的道路上的城市（除了最后一个城市，即该省省会）都必须属于该省。一个城市可以作为多个省的省会。输入 $N-1$ 条边，每条边指定两个数作为被其连接的两个城市的编号。

#### 【代码实现】

```

// 陈锋
#include <iostream>
#include <stack>
#include <vector>
using namespace std;

typedef long long LL;
const int NN = 1000 + 4;
vector<int> G[NN];
stack<int> S;
int N, B, BCnt, BId[NN], Cap[NN]; // 块的个数, 每个点所属块编号, 每个块的中心

void dfs(int u, int fa) {
    size_t sz = S.size();
    for (auto v : G[u]) {
        if (v == fa) continue;
        dfs(v, u);
        if (S.size() >= sz + B) { // 新增点可以分块
            Cap[++BCnt] = u; // 新增块中心点为 u
            while (S.size() > sz) BId[S.top()] = BCnt, S.pop();
        }
    }
    S.push(u);
    if (u == 1) // 注意: root 特殊处理, 未分块的点都放入以 root 为中心的块
        while (!S.empty()) BId[S.top()] = BCnt, S.pop();
}

int main() {
    ios::sync_with_stdio(false), cin.tie(nullptr);
    cin >> N >> B, BCnt = 0;
    for (int i = 1, u, v; i < N; i++) {
        cin >> u >> v;
        G[u].push_back(v), G[v].push_back(u);
    }
    dfs(1, -1);
    cout << BCnt << endl;
    for (int i = 1; i <= N; i++) cout << BId[i] << (i == N ? "\n" : " ");
    for (int i = 1; i <= BCnt; i++) cout << Cap[i] << (i == BCnt ? "\n" : " ");
    return 0;
}
/*
算法分析请参考:《算法竞赛入门经典——训练指南》升级版 6.2 节例题 7
同类题目: Tree, HDU 6394
*/

```

**例 4-52 【树上带修改莫队】糖果公园（WC 2013，牛客 NC 200517）**

有一座糖果公园，里面有 $N$  ( $N \leq 10^5$ ) 个糖果发放点，形成一棵 $N$ 个结点的树，每个结点 $i$ 提供一种编号为 $C_i$ 的糖果，共有 $M$ 种糖果。每种糖果 $i$ 都有一个美味指数 $V_i$ ，一个游客如果重复品尝同一种糖果，会感觉腻味，所获得的愉悦指数也会降低。总的来说，游客第 $i$ 次在发放点 $i$ 品尝糖果 $j$ 的话，愉悦指数 $H$ 增加 $V_j W_i$ ，而一个游客走过树上一条道路所获得的愉悦指数 $H$ 就是该道路上所有 $V_j W_i$ 的总和。当然，公园管理方可能会修改某个发放点上的糖果种类。

给出 $Q$ 个操作，每个操作包含3个整数Type,  $x$ ,  $y$ ，则：

- ☐ Type为0，则 $1 \leq x \leq n$ ， $1 \leq y \leq m$ ，表示将发放点 $x$ 的糖果类型改为 $y$ 。
- ☐ Type为1，则 $1 \leq x, y \leq n$ ，表示计算并输出由出发点 $x$ 到终点 $y$ 的路线上的愉悦指数。

**【代码实现】**

```

// 陈锋
#include <bits/stdc++.h>
using namespace std;
typedef long long LL;
const int NN = 100010, LN = 20;
struct Edge {
    int v, next;
} ES[NN * 2];
int Next[NN], EC = 0;
void add_edge(int x, int y) {
    ES[++EC] = {y, Next[x]};
    Next[x] = EC;
}
stack<int> Stk;
int Dep[NN], fa[NN][LN], // LCA
    BlkId[NN], BLOCK, BlkCnt = 0, // Block
    C[NN], CNT[NN], VIS[NN], MC = 0, QC = 0, CBak[NN]; // 莫队
LL V[NN], W[NN], CurAns = 0, Ans[NN];
void dfs(int x) { // 预处理 LCA 以及树的分块
    Dep[x] = Dep[fa[x][0]] + 1;
    for (int i = 1; (1 << i) <= Dep[x]; i++) fa[x][i] = fa[fa[x][i - 1]][i - 1];
    size_t now = Stk.size();
    for (int i = Next[x]; i; i = ES[i].next) {
        int v = ES[i].v;
        if (v == fa[x][0]) continue;
        fa[v][0] = x;
        dfs(v);
        if (Stk.size() - now > BLOCK) { // 树分块
            BlkCnt++;
            while (Stk.size() > now) BlkId[Stk.top()] = BlkCnt, Stk.pop();
        }
    }
    Stk.push(x);
    if (x == 1) {
        BlkCnt++;
        while (!Stk.empty()) BlkId[Stk.top()] = BlkCnt, Stk.pop();
    }
}

```

```

    }
}

int LCA(int x, int y) { // 倍增 LCA
    if (Dep[x] < Dep[y]) swap(x, y);
    for (int i = LN; i >= 0; i--)
        if ((1 << i) <= Dep[x] - Dep[y]) x = fa[x][i];
    if (x == y) return x;
    for (int i = LN; i >= 0; i--)
        if (Dep[x] >= (1 << i) && fa[x][i] != fa[y][i]) x = fa[x][i], y = fa[y][i];
    return fa[x][0];
}

struct Modify {
    int pos, color, old_color;
} MS[NN];
struct Query {
    int x, y, time, id;
    bool operator<(const Query& b) const {
        if (BlkId[x] != BlkId[b.x]) return BlkId[x] < BlkId[b.x];
        if (BlkId[y] != BlkId[b.y]) return BlkId[y] < BlkId[b.y];
        return time < b.time;
    }
} QS[NN];

inline void add_pos(int u) { CurAns += W[++CNT[C[u]]] * V[C[u]]; } // 莫队
inline void del_pos(int u) { CurAns -= W[CNT[C[u]]--] * V[C[u]]; }
inline void modify(int u, int c) { // 改变 u 的颜色
    if (VIS[u])
        del_pos(u), C[u] = c, add_pos(u);
    else
        C[u] = c;
}

inline void invert(int u) { // 切换 u 的选中状态
    if (VIS[u])
        VIS[u] = 0, del_pos(u);
    else
        VIS[u] = 1, add_pos(u);
}

inline void mark(int u, int l) { // mark [u->l], l 是 u 的祖先
    for (int x = u; x != l; x = fa[x][0]) invert(x);
}

int main() {
    int N, M, Q;
    scanf("%d%d%d", &N, &M, &Q);
    BLOCK = (int)(pow(N, 2.0 / 3) / 2);
    for (int i = 1; i <= M; i++) scanf("%lld", &V[i]);
    for (int i = 1; i <= N; i++) scanf("%lld", &W[i]);
    for (int i = 1, u, v; i < N; i++)

```

```

    scanf("%d%d", &u, &v), add_edge(u, v), add_edge(v, u);
for (int i = 1; i <= N; i++) scanf("%d", &C[i]), CBak[i] = C[i];

dfs(1);
for (int i = 1, op; i <= Q; i++) {
    scanf("%d", &op);
    if (op == 0) {
        Modify& m = MS[++MC];
        scanf("%d%d", &m.pos, &m.color);
        m.old_color = C[m.pos], C[m.pos] = m.color;
    } else {
        Query& q = QS[++QC];
        scanf("%d%d", &q.x, &q.y), q.time = MC, q.id = QC;
        if (BlkId[q.x] > BlkId[q.y]) swap(q.x, q.y); // y 的移动就会少一半
    }
}
sort(QS + 1, QS + 1 + QC);
for (int i = 1; i <= N; i++) C[i] = CBak[i];

int X = 1, Y = 1;
QS[0].time = 0; // [X, Y]: 包含当前结果的区间
for (int i = 1; i <= QC; i++) { // 维护 X, Y 路径上不包含 lca(X, Y) 的点的贡献和
    const Query& q = QS[i];
    int l = LCA(q.x, X), l2 = LCA(q.y, Y), last = QS[i - 1].time;
    mark(X, l), mark(q.x, l), mark(Y, l2), mark(q.y, l2); // X→q.x, Y→q.y
    X = q.x, Y = q.y, l = LCA(X, Y);
    invert(l);
    for (int t = last + 1; t <= q.time; t++) modify(MS[t].pos, MS[t].color);
    for (int t = last; t > q.time; t--) modify(MS[t].pos, MS[t].old_color);
    Ans[q.id] = CurAns;
    invert(l);
}
for (int i = 1; i <= QC; i++) printf("%lld\n", Ans[i]);
}
/*
算法分析请参考:《算法竞赛入门经典——训练指南》升级版 6.2 节例题 8
*/

```

## 本节例题列表

本节讲解的例题及其囊括的知识点，如表4-11所示。

表4-11 嵌套和分块数据结构例题归纳



| 编 号    | 题 号       | 标 题                  | 知 识 点       | 代 码 作 者 |
|--------|-----------|----------------------|-------------|---------|
| 例 4-48 | UVa 11297 | Census               | 二维线段树       | 陈锋      |
| 例 4-49 | UVa 11990 | “Dynamic” Inversion  | 树状数组套静态 BST | 刘汝佳     |
| 例 4-50 | UVa 12003 | Array Transformation | 分块数组        | 刘汝佳     |
| 例 4-51 | NC 208301 | 王室联邦                 | 树分块         | 陈锋      |
| 例 4-52 | NC 200517 | 糖果公园                 | 树上带修改莫队     | 陈锋      |

---

(1) <https://ipsc.ksp.sk/2011/real/problems/g.html>